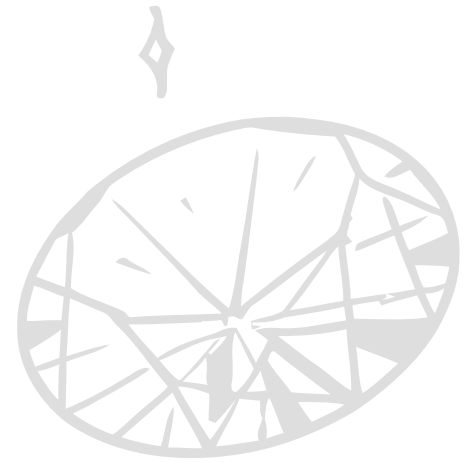
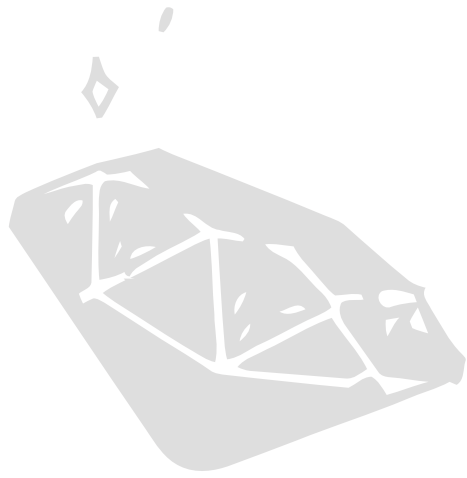
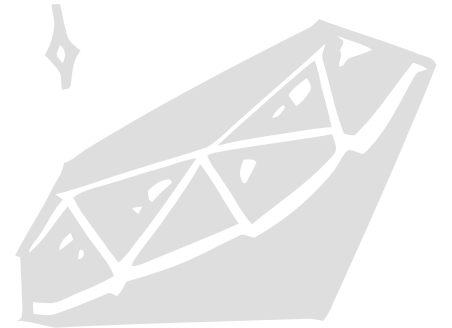
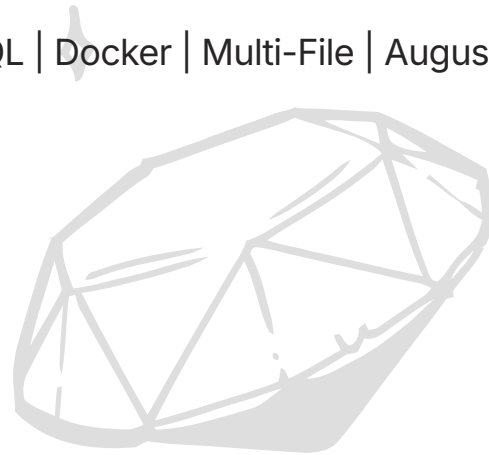
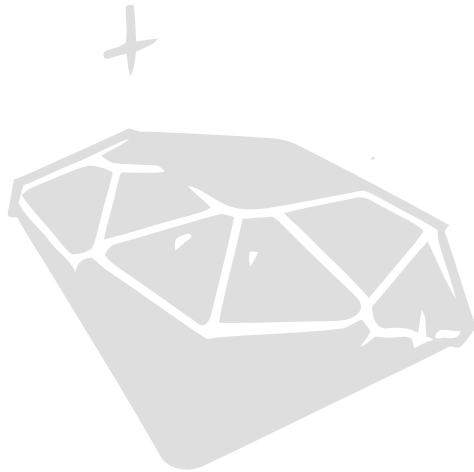




Campus Sport Resources

MySQL C API CRUD + Authentication + Docker

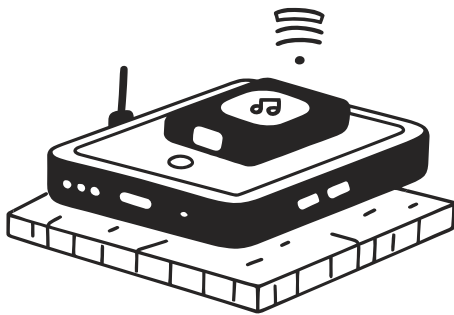
C | MySQL | Docker | Multi-File | August 2026



Overview & Scope

Project overview

A terminal-first CLI service written in C (C99) that performs CRUD operations against a MySQL database using the libmysqlclient C API. Runs in Docker, supports role-based authentication (admin / user), and uses dynamic arrays for in-memory collections mapped to persistent storage.



Scope

In

- CLI CRUD for sport resources
- Dynamic arrays for Resource storage
- Role-based auth (admin, user)
- Docker + Docker Compose deployment

Out

- GUI (not included)
- Password hashing (plain text stored)
- SQL injection protections (not implemented)
- Automated tests (deferred)

Tech Stack



C99

Core application language; standard C features and manual memory management.



libmysqlclient

Native C API for database connections and queries.



Docker & Compose

Containerization and multi-service orchestration (app + mysql).



MySQL 8.0

Persistent relational store; runs in containerized MySQL instance.



GCC

Build toolchain; Makefile orchestrates compilation and linking.



Ubuntu / Alpine

Base images for development and production variants.

Component	Purpose
main.c	Entry point, CLI parsing, program lifecycle
db.h/.c	Connection management, TCP options, query helpers
auth.h/.c	Login, role checks, session context
sport_resource.h/.c	Resource model, CRUD wrappers, array management

Architecture

Container network: app  mysql over TCP. Host maps port 3307 → container 3306. Persistent volume mysql_data stores database files. App container builds with GCC and links libmysqlclient.



Networking

Use DB_HOST="mysql" and mysql_options to force TCP (not socket).



Persistence

Docker volume: mysql_data for durability across restarts.

File Structure

Top-level repository layout — multi-file C project separated by responsibility. Makefile and Docker configuration are first-class artifacts for reproducible builds.

```
campus_sports/  
├── Dockerfile [blue]  
├── docker-compose.yml[blue]  
├── Makefile [blue]  
├── main.c [green]  
├── auth.h / auth.c [green]  
├── db.h / db.c [green]  
└── sport_resource.h / sport_resource.c [green]
```



Data Structures



Resource

```
int id; char  
name[100]; char  
category[50];  
char  
location[100]; int  
quantity; int  
available;
```

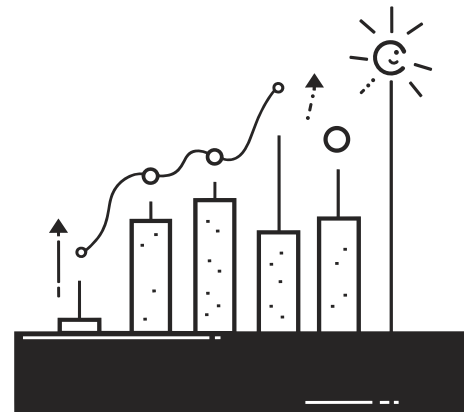


User

```
int id; char  
username[50];  
char  
password[50]; int  
is_admin;
```

Dynamic array (ResourceArray)

Resource *data; int count; int capacity; Grow
strategy: 0 → 4 → 8 → 16 (capacity doubles). Functions: init,
push, remove, free.



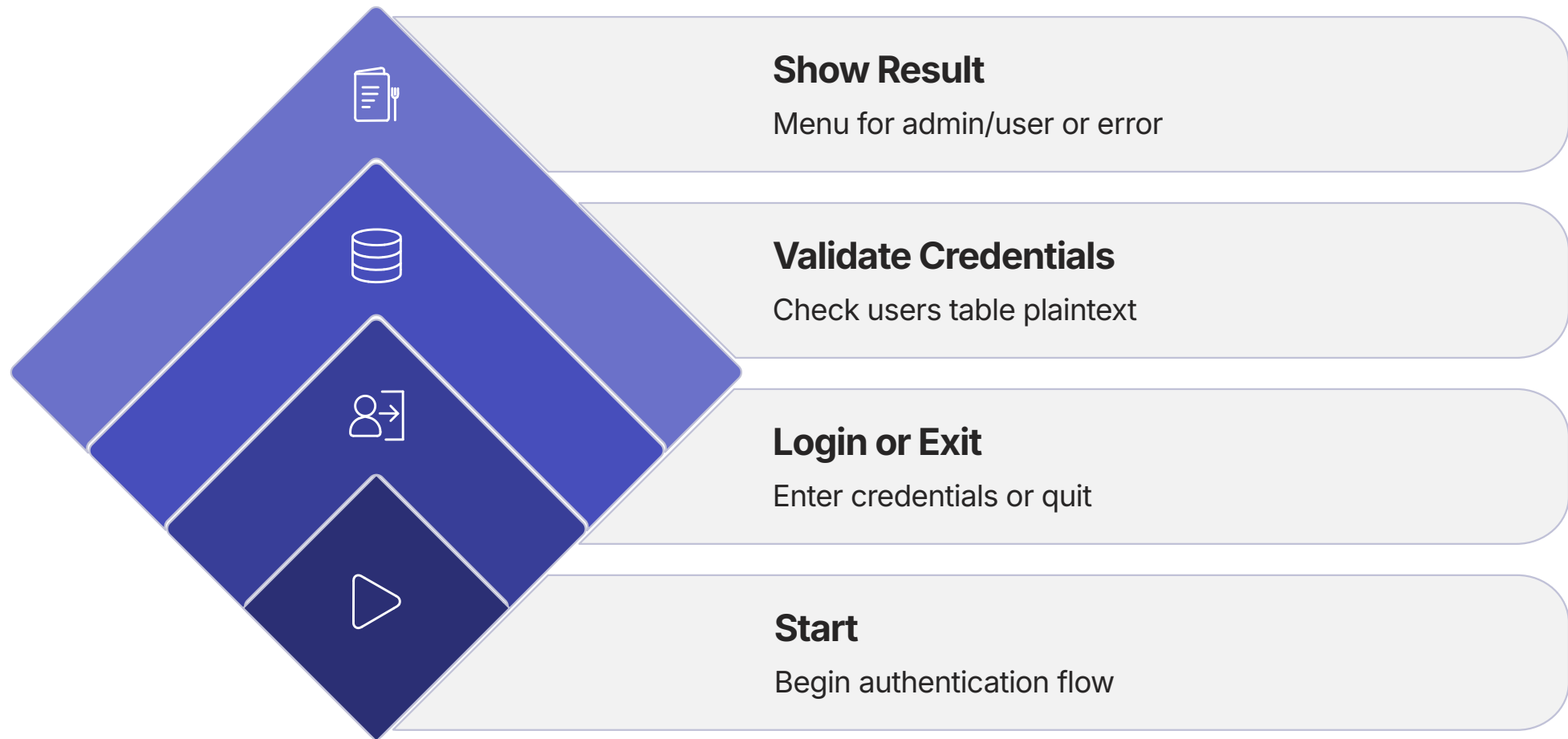
Database Schema

sport_resources	
id	INT PK AUTO_INCREMENT
name	VARCHAR(100)
category	VARCHAR(50)
location	VARCHAR(100)
quantity	INT
available	INT
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP

users	
id	INT PK AUTO_INCREMENT
username	VARCHAR(50) UNIQUE
password	VARCHAR(50)
is_admin	TINYINT(1)
created_at	TIMESTAMP DEFAULT CURRENT_TIMESTAMP

Relationship: **users** —manages—▶ **sport_resources**. Permissions enforced in auth layer: admins full CRUD; users limited to read and borrow/update availability.

Authentication & Flow



Auth uses username/password checks against users table (plain text credentials). Session context stores user id and is_admin flag. Menu options gated by is_admin.

TCP connection fix

BEFORE: client attempted local Unix socket → connection failed. AFTER: set DB_HOST="mysql" and call `mysql_options(MYSQL_OPT_PROTOCOL, "TCP")` to force TCP → successful connection.

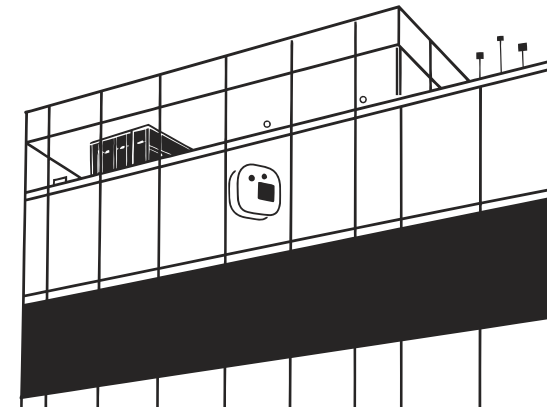
Docker Build & Run

Terminal commands (monospace block shown for copy/paste). Use docker-compose to build and run the multi-container system.

```
$ docker-compose up --build
$ docker-compose down -v
$ docker logs campus_mysql
```

Credentials

Service	Credentials
DB	mysql:3306 · root / campus123 · database: campus_sports
App (sample)	admin / admin123 · user / user123



❏ Use host port mapping 3307:3306 in docker-compose for local debugging to avoid conflicts with host MySQL.

Limitations & Timeline

Limitation	Impact	Mitigation (future)
Plain text passwords	Security risk	Introduce bcrypt / Argon2, salt storage
No SQL injection protection	Query injection risk	Use prepared statements / parameter binding
Hardcoded credentials	Operational risk	Move to env files / secrets manager

Project timeline (sequence)

1. DB Setup
2. Data Structures
3. CRUD Implementation
4. Auth Layer
5. Dockerize (Compose)
6. TCP Fix & Integration
7. Testing & Hardening (future)